Architectures logicielles et matérielles

Cours 1 : Introduction - Révisions

Laurence Pierre

UGA Université Grenoble Alpes

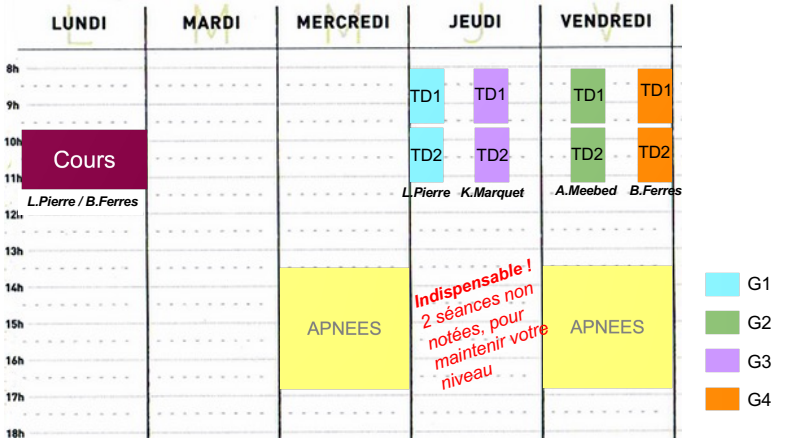
L3 Informatique 2026-2027

Préambule - informations

- Slides de cours (avec trous) et sujets de TD/TP seront dans le Moodle
  - **Pas** de distribution de corrigés types des TD/TP
  - Si questions :
    - Généralités, organisation,… : Laurence.Pierre@univ-grenoble-alpes.fr
    - Questions techniques sur TD/TP : mail à vos intervenants
- Contrôle des connaissances
  - Contrôle continu : 2 quicks notés
    - Cartes et entrées/sorties (23/10)
    - Conception : circuits séquentiels, machines à états (20/11)
  - Examen (2h30) - ATTENTION : pas de règle du max en ALM



Préambule - organisation



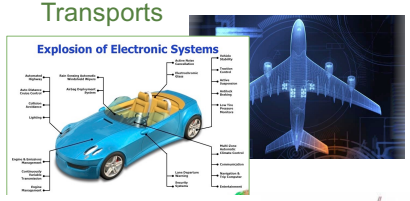
Préambule - informations

- **Présence obligatoire** en TD. Préparation :
  - TD réalisés par groupes de 3 ou 4, TP réalisés en binôme
  - **Questions préliminaires** au début de chaque planche de TD, à travailler individuellement **avant** le TD
  - **Rédaction** commune, par groupe, des réponses aux questions préliminaires (rédacteur différent chaque semaine), **remise à 8h15**
  - A la **fin** du TD : **remise** d'une question (d'un exo) du TD
- **Présence obligatoire** en TP. Préparation :
  - Au début des TP : **rédaction** commune, par groupe, de la **compréhension** du sujet du TP, **remise avant** de commencer le TP

Motivations du cours - besoins

- Systèmes électroniques dans tous les domaines…

Transports



https://www.chipestimate.com/IPs-Silent-Presence-in-Automotive-Market/blogs/92

Médical



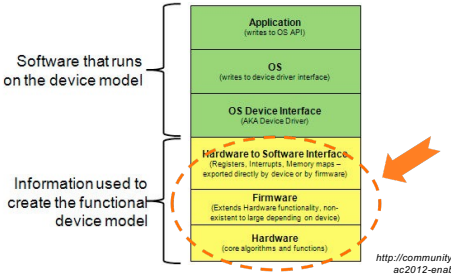
IoT, cloud,…



https://www.elprocus.com/understanding-android-based-home-automation-systems/

Motivations du cours - besoins

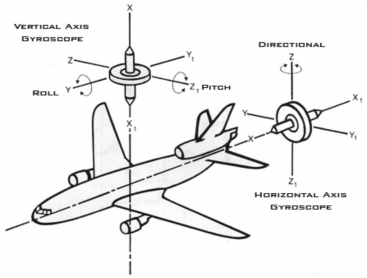
- Systèmes électroniques dans tous les domaines…
  - Transports, médical, maison intelligente, cloud, IoT,…
- Conception **matérielle/logicielle** ➡ Programmer pour ces systèmes embarqués nécessite une compréhension du **matériel** et de **l'interface matériel/logiciel**.



http://community.cadence.com/cadence_blogs_8/b/sd/archive/2012/06/26/dac2012-enabling-the-programming-of-an-extensible-processing-platform

Motivations du cours

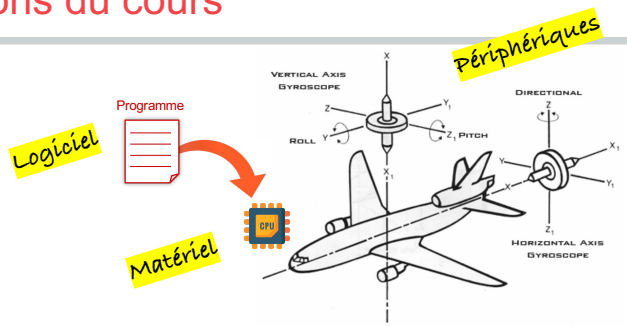
- Exemple



crédit photo : http://www.dutchops.com

Motivations du cours

- Exemple

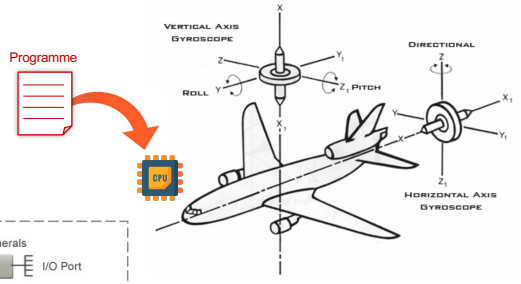


crédit photo : http://www.dutchops.com

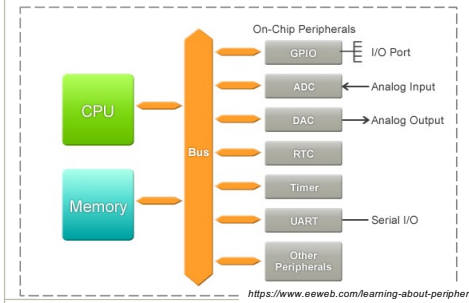
  - Comprendre la conception logicielle n'est pas suffisant !

Motivations du cours

- Exemple



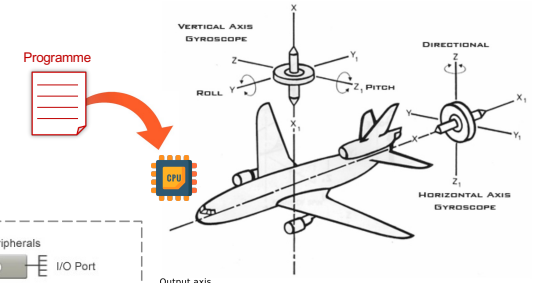
crédit photo : http://www.dutchops.com



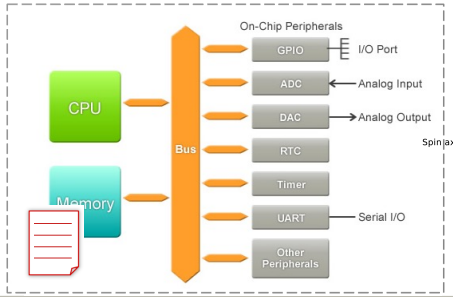
https://www.eeweb.com/learning-about-peripherals-part-1-gpio/

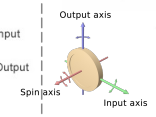
Motivations du cours

- Exemple



crédit photo : http://www.dutchops.com



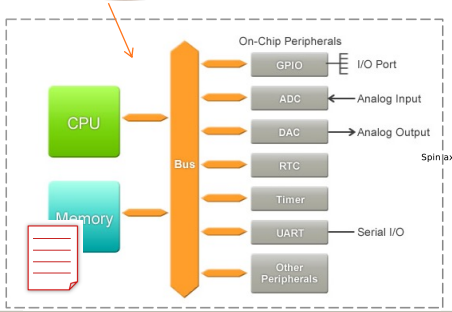


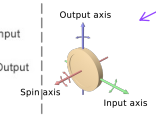
Motivations du cours

- Objectifs de ce cours : étudier les bases pour répondre principalement à deux questions

2 *Principes de base pour **concevoir le matériel** ?*

1 *Comment logiciel et CPU gèrent-ils les **interactions avec les périphériques** ?*





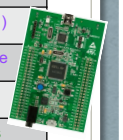
Objectifs du cours / prérequis

- Objectifs : étudier les bases pour répondre à
  - 1 Comment logiciel et CPU interagissent-ils avec les périphériques ?
    - Adressage, accès aux périphériques, utilisation d'interruptions, gestion des interruptions par le processeur,...
  - 2 Comment le matériel est-il conçu ?
    - Principes de base pour la conception de circuits combinatoires, séquentiels, avec partie contrôle et partie opérative
- **Prérequis**
  - **INF401** (L2) : codage, assembleur, organisation processeur et principes d'interprétation des instructions 1 2
  - Programmation en C (INF203, INF304, mise à niveau pré-L3) 1
  - Algèbre de Boole (INF402) 2

Planning prévisionnel

E/S, programmation

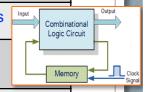
	Cours	TD/TP 1	TD/TP 2
1	7/9 Introduction, révisions	TD révisions codages	TD révisions op. bas niveau
2	14/9 Microcontrôleur / périphériques	TD périphériques, adresses	**TP** révisions (masques,…)
3	21/9 Carte, E/S, ports	TD prog. de base sur carte	**TP** prog. de base sur carte
4	28/9 Communic. interruptions/polling	**TP** polling (RNG)	**TP** interruptions (RNG)
5	5/10 Processeur et interruptions	TD proc. et interruptions	TD vecteur d'interruptions
6	12/10 Protocole SPI (et I2C)	**TP** SPI (accéléromètre)	**TP** SPI fin
7			
8			
9			
10			
11			



Planning prévisionnel

Conception de base

	Cours	TD/TP 1	TD/TP 2
1			
2			
3			
4			
5			
6			
7	19/10 Circuits combinatoires	TD circuits combinatoires simples	TD circuits arithmétiques
8	2/11 Circuits séquentiels, FSM	TD circuits séquentiels simples	TD circuits séquentiels
9	9 /11 Conception avec PC + PO (début)	TD1 FSM	TD2 FSM
10	16/11 Concept. avec PC + PO (suite)	**TP** conception PC + PO	**TP** conception PC + PO fin
11	23/11 Compléments / révisions	TD1 conception PC + PO	TD2 conception PC + PO



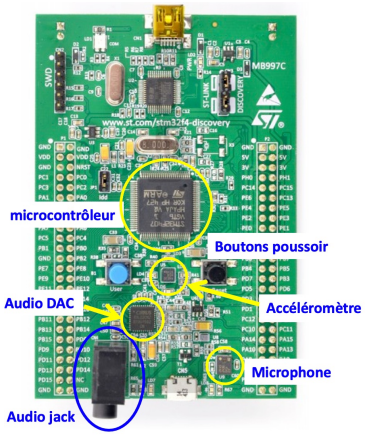
Planning prévisionnel

	Cours	TD/TP 1	TD/TP 2
1	7/9 Introduction, révisions	TD révisions codages	TD révisions op. bas niveau
2	14/9 Microcontrôleur / périphériques	TD périphériques, adresses	**TP** révisions (masques,…)
3	21/9 Carte, E/S, ports	TD prog. de base sur carte	**TP** prog. de base sur carte
4	28/9 Communic. interruptions/polling	**TP** polling (RNG)	**TP** interruptions (RNG)
5	5/10 Processeur et interruptions	TD proc. et interruptions	TD vecteur d'interruptions
6	12/10 Protocole SPI (et I2C)	**TP** SPI (accéléromètre)	**TP** SPI fin
7	19/10 Circuits combinatoires	TD circuits combinatoires simples	TD circuits arithmétiques
8	2/11 Circuits séquentiels, FSM	TD circuits séquentiels simples	TD circuits séquentiels
9	9 /11 Conception avec PC + PO (début)	TD1 FSM	TD2 FSM
10	16/11 Concept. avec PC + PO (suite)	**TP** conception PC + PO	**TP** conception PC + PO fin
11	23/11 Compléments / révisions	TD1 conception PC + PO	TD2 conception PC + PO

Révisions pour programmation avec E/S sur carte

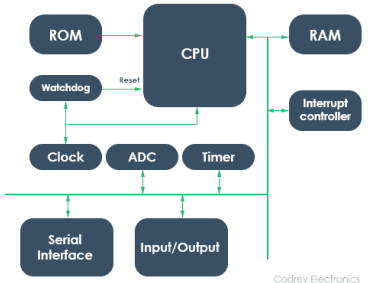
Contexte des futurs TP

- Nous programmerons ici en C sur carte STM32F4



Contexte des futurs TP

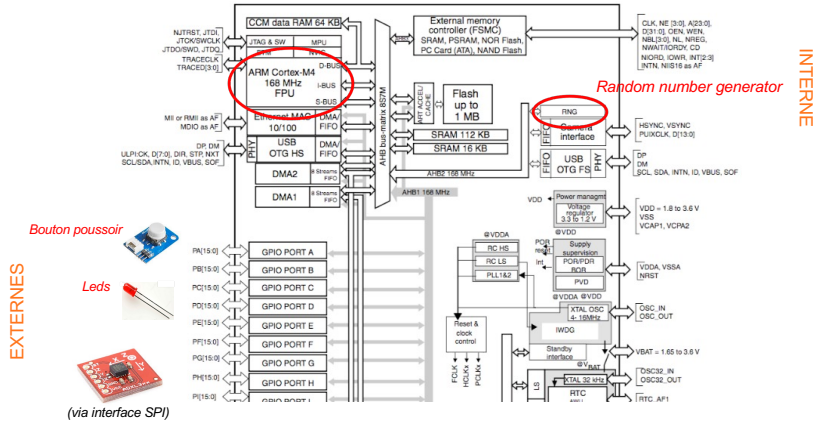
- Microcontrôleur…
  - Un **microcontrôleur** contient généralement un CPU (ici ARM Cortex M4), de la mémoire, des **périphériques** internes (ex. timer, RNG), et des entrées/sorties et interfaces série pour d'autres **périphériques**



https://www.codrey.com/learn/microcontroller-basics/

Contexte des futurs TP

- Microcontrôleur… (carte STM32F4)



Contexte des futurs TP

- Microcontrôleur…
  - Il nous faudra interagir (en C) avec
    - Les **leds** (hors du microcontrôleur)
    - Le générateur de nombres aléatoires, ***random number generator*** (dans le microcontrôleur)
    - L'**accéléromètre** (interface SPI, *Serial Peripheral Interface*)
    - Un périphérique avec interface I2C (*Inter-Integrated Circuit*)
- Pour ce faire, il faut avant tout maîtriser quelques bases
  - Les codages : binaire, hexadécimal
  - Les opérations bit à bit, masquages,…
  - Les manipulations d'adresses, de pointeurs, les cast,…

1 Rappels sur les codages

- Codage des entiers non signés, sur n digits

$x_{n-1}\ x_{n-2} \dots x_2\ x_1\ x_0$

digit de poids fort — digit de poids faible

  - Binaire (base B=2) : chaque digit x_k peut valoir 0 ou 1. Ils sont appelés des "bits" (binary digit)
  - Octal (base B=8) : chaque digit x_k peut valoir 0, 1, 2,… ou 7
  - Hexadécimal (base B=16) : chaque digit x_k peut valoir 0, 1, 2,…, 8, 9, A, B, C, D, E, ou F

- L'interprétation entière est $X = \sum_{k=0}^{n-1} x_k B^k$

Rappels sur les codages

- Codage des entiers non signés, sur n digits, en binaire
  - Exemple :
    - Valeur binaire = 00000011 10110011

      $X = 1.2^0 + 1.2^1 + 0.2^2 + 0.2^3 + 1.2^4 + 1.2^5 + 0.2^6 + 1.2^7 + 1.2^8 + 1.2^9 + 0.2^{10} + \dots + 0.2^{15} = 947$ en décimal
- Codage des entiers non signés en binaire, **interprétation en hexadécimal**
  - Chaque "paquet" de 4 bits représente un nombre entre 0 et 15, soit un digit hexadécimal => il suffit d'interpréter par "paquets" de 4 bits !!
  - Exemple :
    - Valeur binaire = 0 3 B 3 : 0000 0011 1011 0011

      X = 03B3 en hexadécimal

Rappels sur les codages

- **Hexadécimal** : on pourra remarquer que
  - $15 = 2^4-1 = 00001111$ en binaire = 0x0F en hexadécimal
  - $16 = 2^4 = 00010000$ en binaire = 0x10 en hexadécimal
  - $255 = 2^8-1 = 11111111$ en binaire = 0xFF en hexadécimal
  - $256 = 2^8 = 100000000$ en binaire = 0x100 en hexadécimal
  - etc...
- et aussi qu'on peut sommer directement en hexadécimal
  - Exemple :
    - Calculer la somme de 0000 0011 1011 0011 (947) et 0000 0010 1000 1010 (650)

      => convertir en hexadécimal et sommer digit par digit

?

Rappels sur les codages

- **Hexadécimal** : on pourra remarquer que
  - $15 = 2^4-1 = 00001111$ en binaire = 0x0F en hexadécimal
  - $16 = 2^4 = 00010000$ en binaire = 0x10 en hexadécimal
  - $255 = 2^8-1 = 11111111$ en binaire = 0xFF en hexadécimal
  - $256 = 2^8 = 100000000$ en binaire = 0x100 en hexadécimal
  - etc...
- et aussi qu'on peut sommer directement en hexadécimal
  - Exemple :
    - Calculer la somme de 0000 0011 1011 0011 et 0000 0010 1000 1010

      => on calcule la somme de 03B3 et 028A

Rappels sur les codages

- Le codage des entiers signés se fait couramment en **complément à 2**
  - Les entiers négatifs sont représentés comme suit :
    - on prend la **négation bit à bit** de la représentation de l'entier positif (c'est le complément à 1)
    - puis on **ajoute 1** au résultat
  - NB. En pratique il suffit, en partant de la droite, de conserver tous les 0 jusqu'au premier 1 (compris) et d'inverser ensuite tous les bits jusqu'au bit de poids fort
    - Exemple : représentation de -650

?

2 Opérations bit à bit

- On rappelle que C propose les **opérateurs bit à bit** :
  - **et** bit à bit &
  - **ou** bit à bit |
  - complément à 1 (**négation** bit à bit) ~
  - **ou exclusif** bit à bit ^
- Chaque bit de l'entier résultant correspond à l'opération booléenne sur les bits du même rang des opérandes
  - Exemples

```
  1010        1010        1010
& 0011      | 0011      ^ 0011
------      ------      ------
  0010        1011        1001
```


Et décalages…

- C propose également des **opérateurs de décalage**
  - **décalage logique à gauche** (logic shift left) de k bits, **x << k** : les bits de x sont décalés de k positions sur la gauche (suppression des bits sortants), et insertion de k bits de valeur 0 à droite
  - **décalage logique à droite** (logic shift right) de k bits, **x >> k** : les bits de x sont décalés de k positions sur la droite (suppression des bits sortants), et insertion de k bits de valeur 0 à gauche
- Note. Les opérateurs bit à bit ne doivent être utilisés qu'avec des entiers non signés (sinon le comportement est implémentation-dépendant)
  - => l'opération de décalage arithmétique à droite n'est pas considérée présente en C

Réalisation/utilisation de masques

- Les opérateurs bit à bit et décalages sont utiles pour créer et appliquer des masques
  - Les masques permettent de **consulter** (ou **modifier**) la valeur d'un seul bit dans un entier naturel, ou d'un sous-bloc de bits
- Pour **consulter** uniquement la valeur du bit numéro k dans un entier naturel X, le principe est le suivant :
  - créer un "**masque**" dont tous les bits sont à 0 sauf le bit n°k
  - "appliquer" ce masque à X, c'est à dire faire leur **et** bit à bit

$$\begin{array}{cc} & x_{n-1}\ x_{n-2}\ \cdots\ x_k\ \cdots\ x_2\ x_1\ x_0 \\ \& & 0\ \ 0\ \ldots\ \ 1\ \ldots\ 0\ \ 0\ \ 0 \end{array}$$

*Créer ceci avec **décalage***

Analyse du résultat :
un résultat nul indique que x_k est à 0, sinon x_k est à 1

Réalisation/utilisation de masques

- Les opérateurs bit à bit et décalages sont utiles pour créer et appliquer des masques
  - Les masques permettent de **consulter** (ou **modifier**) la valeur d'un seul bit dans un entier naturel, ou d'un sous-bloc de bits
- Pour **consulter** uniquement la valeur du bit numéro k dans un entier naturel X :
  - Exemple : valeur du bit numéro 5 de 0x3B3

```
  0 0 0 0 0 0 1 1 1 0 1 1 0 0 1 1
& 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0   <-- 1 << 5
-------------------------------
  0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
```


Réalisation/utilisation de masques

- Les opérateurs bit à bit et décalages sont utiles pour créer et appliquer des masques
  - Les masques permettent de **consulter** (ou **modifier**) la valeur d'un seul bit dans un entier naturel, ou d'un sous-bloc de bits
- Pour **modifier** uniquement la valeur du bit numéro k dans un entier naturel X, le principe est le suivant :
  - **remettre à 0** le bit n°k (en utilisant un masque dont seul ce bit est à 0)
  - puis **placer la valeur voulue** v (0 ou bien 1) dans X en utilisant un **ou** bit à bit

Réalisation/utilisation de masques

- Les opérateurs bit à bit et décalages sont utiles pour créer et appliquer des masques
  - Les masques permettent de **consulter** (ou **modifier**) la valeur d'un seul bit dans un entier naturel, ou d'un sous-bloc de bits
- Pour **modifier** uniquement la valeur du bit numéro k dans un entier naturel X :
  - Exemple : mettre la valeur V à l'emplacement du bit numéro 5 de 0x3B3

```
  0 0 0 0 0 0 1 1 1 0 1 1 0 0 1 1
& 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1   <-- ~ (1 << 5)
-------------------------------
  0 0 0 0 0 0 1 1 1 0 0 1 0 0 1 1
| 0 0 0 0 0 0 0 0 0 0 V 0 0 0 0 0   <-- V << 5
-------------------------------
  0 0 0 0 0 0 1 1 1 0 V 1 0 0 1 1
```


Réalisation/utilisation de masques

- Les opérateurs bit à bit et décalages sont utiles pour créer et appliquer des masques
  - Les masques permettent de **consulter** (ou **modifier**) la valeur d'un seul bit dans un entier naturel, ou d'un sous-bloc de bits
- En **exercice** à préparer : on a vu comment modifier uniquement la valeur du bit numéro k dans un entier naturel X, mais
  - est-il nécessaire de procéder de la même façon si l'objectif est de placer une valeur V à l'emplacement du bit numéro k d'une variable entière X, en mettant tous ses autres bits à 0 ?
  - donner l'affectation correspondante

?

3 Adressage, pointeurs

- On rappelle qu'en C, une variable de type **pointeur** sur un élément de type T est déclarée par : ***T *nomvar;***
  - Exemples
    - px est un pointeur sur entier : *int *px;*
    - pc est un pointeur sur caractère : *char *pc;*
- L'opérateur & permet de récupérer **l'adresse** d'une variable, et l'opérateur * permet de désigner **l'objet pointé** (repéré) par le pointeur
  - Exemples : que se passe-t-il ici (les variables sont de quels types, quelles variables sont initialisées, affectées, modifiées ?)

```
int x=5,y=25;
int *p = &x;
int *q;
*p = 3;
p = &y;
*p += 1;
```

?

Adressage, pointeurs

- On rappelle qu'il est également possible de
  - Avoir recours à "l'arithmétique sur pointeurs" : une variable de type pointeur peut être incrémentée (ou décrémentée)

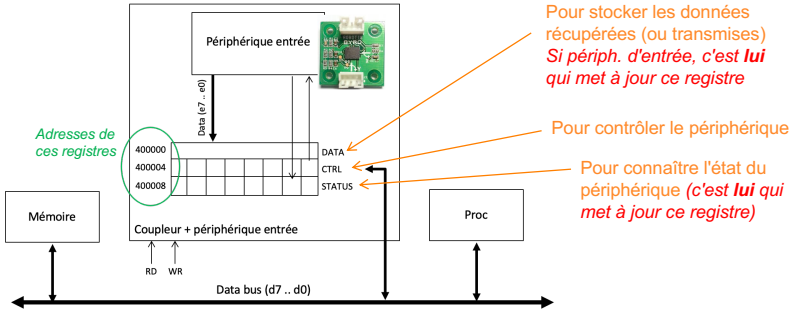
    Attention, une **incrémentation** de pointeur correspond à un déplacement en mémoire **de la taille** (nombre d'octets) **d'un élément pointé**
    - NB. On rappelle que le seul type prédéfini dont la taille est prévisible sur toute machine est le type *char*.
      On pensera à utiliser les **types int8_t, uint8_t, int16_t, etc…**
  - Avoir recours à une conversion de type (cast)
    - Exemple :

```
uint16_t *pz;
pz = (uint16_t *)0x400000;
```

?

Adressage, pointeurs

- Nous approfondirons dans les cours suivants le fait que le processeur est relié aux **périphériques** via des coupleurs (contrôleurs) d'entrées/sorties, par ex. (fictif) :



Pour stocker les données récupérées (ou transmises) *Si périph. d'entrée, c'est **lui** qui met à jour ce registre*

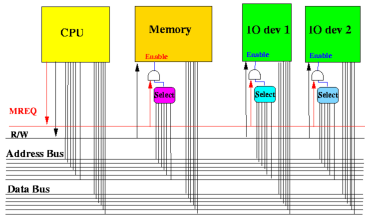
Pour contrôler le périphérique

Pour connaître l'état du périphérique *(c'est **lui** qui met à jour ce registre)*

et qu'il faudra **accéder aux contenus** des registres

Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques** avec lesquels notre programme communiquera
- L'adressage vers les périphériques se fera par memory mapping, c'est-à-dire que
  - Le **même espace d'adressage** est utilisé pour la mémoire et les périphériques
    - des **adresses spécifiques** correspondent aux périph.
    - l'architecture matérielle assure le **décodage** (voir partie 2 du cours)



http://www.cs.emory.edu/~cheung/Courses/355/Syllabus/6-io/addressing.html

Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques** avec lesquels notre programme communiquera
- L'adressage vers les périphériques se fera par memory mapping, c'est-à-dire que
  - Le **même espace d'adressage** est utilisé pour la mémoire et les périphériques (et des *adresses spécifiques* correspondent aux périphériques)
  - => on accédera donc **de la même façon** à la mémoire et aux périphériques

Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques**
- On accède à ces valeurs via les **adresses** de ces éléments mémorisants
  - Exemple : dans l'exemple (fictif) du slide 37
    - les données seront récupérées en **accédant** à l'adresse 0x400000
    - le périphérique sera contrôlé en **modifiant** des bits dans la valeur se trouvant à l'adresse 0x400004 (chaque bit joue un rôle spécifique, voir dans les prochains cours/TP)
    - on connaîtra l'état du périphérique en **accédant** à la valeur se trouvant à l'adresse 0x400008 (chaque bit donne une information)

Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques**
- On accède à ces valeurs via les **adresses** de ces éléments mémorisants. Il est important de **préciser le type** des données accédées.
  - Exemple : dans l'exemple (fictif) du slide 37
    - Que signifient les macros suivantes :

```
#define DATA_ADR (uint8_t *)0x400000
#define CTRL_ADR (uint8_t *)0x400004
#define STATUS_ADR (uint8_t *)0x400008
```

?

Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques**
- On accède à ces valeurs via les **adresses** de ces éléments mémorisants. Il est important de **préciser le type** des données accédées.
  - Exemple : dans l'exemple (fictif) du slide 37
    - Et on qualifiera même comme suit :

```
#define DATA_ADR (volatile uint8_t *)0x400000
#define CTRL_ADR (volatile uint8_t *)0x400004
#define STATUS_ADR (volatile uint8_t *)0x400008
```

Le contenu mémoire peut être modifié par une entité extérieure (le périphérique lui-même)

Adressage, pointeurs

- Note. Exemple utilisation de volatile

```
int main(void) {
  int32_t *p, *i;
  int32_t x;
  p = &var1;
  i = &var2;
  while (*i != 0) {
     x = *p;
  }
  return x;
}
```

Compilation, en O1 (extrait)

```
80001f0:    4b02      ldr     r3, [pc, #8]
80001f2:    681b      ldr     r3, [r3, #0]
80001f4:    b103      cbz     r3, 80001f8
80001f6:    e7fe      b.n     80001f6
80001f8:    2000      movs    r0, #0
80001fa:    4770      bx      lr
```

```
int main(void) {
  volatile int32_t *p, *i;
  int32_t x;
  p = &var1;
  i = &var2;
  while (*i != 0) {
     x = *p;
  }
  return x;
}
```

Compilation, en O1 (extrait)

```
80001f0:    4b05      ldr     r3, [pc, #20]
80001f2:    681b      ldr     r3, [r3, #0]
80001f4:    b12b      cbz     r3, 8000202
80001f6:    4b04      ldr     r3, [pc, #16]
80001f8:    6858      ldr     r0, [r3, #4]
80001fa:    681a      ldr     r2, [r3, #0]
80001fc:    2a00      cmp     r2, #0
80001fe:    d1fb      bne.n   80001f8
8000200:    4770      bx      lr
8000202:    2000      movs    r0, #0
8000204:    4770      bx      lr
```


Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques**
- On accède à ces valeurs via les **adresses** de ces éléments mémorisants. Il est important de **préciser le type** des données accédées.
  - Exemple : dans l'exemple (fictif) du slide 37
    - Et pour l'accès aux données…

```
#define DATA_ADR (volatile uint8_t *)0x400000
#define CTRL_ADR (volatile uint8_t *)0x400004
#define STATUS_ADR (volatile uint8_t *)0x400008
...
uint8_t x;
```

← *Comment placer la donnée x à l'adresse DATA_ADR* ?

Adressage, pointeurs

- Il nous faudra **consulter ou modifier des valeurs** présentes dans les éléments mémorisants (registres) associés aux **périphériques**
- On accède à ces valeurs via les **adresses** de ces éléments mémorisants. Il est important de **préciser le type** des données accédées.
  - En **exercice** à préparer : dans l'exemple (fictif) du slide 37
    - Comment faire dans ce cas ?

```
#define DATA *((volatile uint8_t *)0x400000)
#define CTRL *((volatile uint8_t *)0x400004)
#define STATUS *((volatile uint8_t *)0x400008)
...
uint8_t x;
```

← *Comment placer la donnée x dans DATA* ?